Cloud-Edge System for Scheduling Unpredictable LLM Requests With Combinatorial Bandit

Yandi Li, Jianxiong Guo, *Member, IEEE*, Zhiqing Tang, *Member, IEEE*, Xingjian Ding, Juncheng Wang, *Member, IEEE*, Tian Wang, *Senior Member, IEEE*, and Weijia Jia, *Fellow, IEEE*

***Abstract*—The rapid growth in demand for large language models (LLMs) has strained cloud-edge infrastructure. While edges offer low latency and clouds provide vast resources, scheduling LLM requests efficiently remains a major challenge due to their unpredictable processing times, which leads to Head-of-Line (HOL) blocking that degrades system throughput and responsiveness. To address this, we introduce the Online Cloud-Edge Collaborative Request Scheduling (OCE-CRS) framework. OCE-CRS models the proactive scheduling of LLM requests as a contextual combinatorial bandit problem. At its core is our novel Combinatorial Neural Delayed Upper Confidence Bound (CN_DUCB) algorithm, which learns to predict request processing times from the semantic content of the request prompt alone. This enables an inspired policy based on Shortest Job First (SJF) that prioritizes shorter jobs for edge execution, simultaneously maximizing throughput and mitigating HOL blocking. To prevent time-consuming neural network training from blocking scheduling decisions, we employ an asynchronous mechanism. This decouples model updates from the real-time scheduling loop, effectively handling the resultant delayed feedback where observations from past rounds are used in later training steps. We provide a theoretical sublinear regret bound for our algorithm. Extensive experiments validate that OCE-CRS significantly improves throughput, Job Completion Time (JCT), and queueing delay, demonstrating superior performance and robustness in both static and continuous batching environments.**

Received 22 June 2025; revised 26 August 2025; accepted 15 September 2025. Date of publication 18 September 2025; date of current version 11 December 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62202055, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2025A1515012843, in part by the Start-up Fund from Beijing Normal University under Grant 312200502510, in part by the Internal Fund from Beijing Normal-Hong Kong Baptist University under Grant UICR0400003-24 and Grant UICR0200022-25, and in part by the Interdisciplinary Intelligence SuperComputer Center of Beijing Normal University (Zhuhai). *(Corresponding author: Jianxiong Guo.)*

Yandi Li is with the Hong Kong Baptist University, Kowloon Tong Hong Kong, and also with the Guangdong Key Lab of AI and Multi-Modal Data Processing, Department of Computer Science, Beijing Normal-Hong Kong Baptist University, Zhuhai 519087, China (e-mail: liyandi@uic.edu.cn).

Jianxiong Guo and Weijia Jia are with the Advanced Institute of Natural Sciences, Beijing Normal University, Zhuhai 519087, China, and also with the Guangdong Key Lab of AI and Multi-Modal Data Processing, Beijing Normal-Hong Kong Baptist University, Zhuhai 519087, China (e-mail: jianxiongguo@bnu.edu.cn; jiawj@bnu.edu.cn).

Zhiqing Tang and Tian Wang are with the Advanced Institute of Natural Sciences, Beijing Normal University, Zhuhai 519087, China (e-mail: zhiqingtang@bnu.edu.cn; tianwang@bnu.edu.cn).

Xingjian Ding is with the School of Computing, Beijing University of Technology, Beijing 100124, China (e-mail: dxj@bjut.edu.cn).

Juncheng Wang is with the Department of Computer Science, Hong Kong Baptist University, Kowloon Tong Hong Kong (e-mail: jcwang@comp.hkbu.edu.hk).

https://github.com/myfilezone/OCE-CRS.

This article has supplementary downloadable material available at https://doi.org/10.1109/TSC.2025.3611379, provided by the authors.

Digital Object Identifier 10.1109/TSC.2025.3611379

***Index Terms*—Large language models (LLMs), request scheduling, cloud-edge collaboration, combinatorial bandit, delayed feedback, performance optimization.**

I. Introduction

THE proliferation of Large Language Models (LLMs) across diverse applications has placed unprecedented strain on existing computing infrastructure, spanning from resource-constrained edge devices to powerful cloud data centers. Edge computing offers the advantage of low-latency processing due to user proximity but is often limited in computational power and memory [1], [2]. Conversely, the cloud provides vast, scalable resources but introduces significant communication latency. Cloud-edge collaborative systems aim to synergize these environments, creating a hybrid model crucial for delivering the high performance and responsiveness users expect from modern AI services [3], [4]. An intuitive and effective strategy in such a system is to process latency-sensitive or simpler tasks at the edge while offloading computationally intensive workloads to the cloud [5], [6], [7], [8].

However, implementing this strategy for LLM requests reveals a multi-layered scheduling challenge that hinges on the ability to consistently differentiate between "simple" and "complex" requests. The first layer is the macro-level decision of workload distribution: to protect the limited resources of the edge, it is imperative to identify and locally execute simple (i.e., fast-processing) requests while offloading more complex ones to the cloud. The second, more granular layer involves scheduling both the edge and the cloud. Unlike traditional tasks with predictable workloads [9], [10], LLM inference time is highly unpredictable. This unpredictability stems primarily from the autoregressive nature of token generation, where the total number of output tokens, a key determinant of processing time, is unknown beforehand. Consequently, a simple First-Come-First-Served (FCFS) policy often leads to severe Head-of-Line (HOL) blocking: a complex request requiring lengthy processing can occupy resources and delay numerous subsequent, potentially simpler and shorter requests. This significantly degrades system performance and quality of service [11]. To address both layers of this problem, it is essential to move towards a Shortest Job First (SJF) scheduling paradigm. This requires defining request "complexity" by its processing time and, critically, developing a method to predict this time for each request before execution.

The task of prediction is itself non-trivial. The processing time of an LLM request is not a fixed property but is influenced by a complex interplay of factors. These include not only the prompt's complexity but also the specific LLM size, the underlying hardware (e.g., CPU versus GPU), dynamic batching

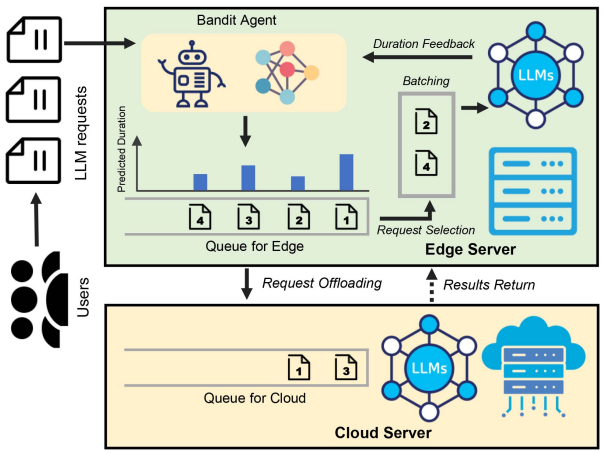


Fig. 1. Overview of the online cloud-edge collaborative request scheduling (OCE-CRS) framework. LLM requests from users are intercepted by a bandit agent on the edge server. The agent predicts the processing duration for each request, selecting a batch of predicted shorter requests for local execution and offloading the remaining longer requests to the cloud server. The actual processing time from the edge provides duration feedback to the agent, enabling online learning and continuous improvement of scheduling decisions.

effects, and KV cache management. This means that the distribution pattern of processing times can shift dramatically from one deployment environment to another, rendering any offline, pre-trained prediction model ineffective and non-generalizable. Furthermore, even within a single, stable environment, instantaneous system state fluctuations can introduce stochasticity to processing times. This necessitates a robust mechanism that can continuously learn and adapt to changing conditions. The ideal solution, therefore, is an online learning approach that can dynamically estimate processing times by learning from incoming requests. Such an approach must balance exploitation (using the current knowledge to make the best scheduling decision) with exploration (processing uncertain requests to refine its knowledge), thereby adapting to the unique and evolving characteristics of the specific deployment.

To tackle these challenges, we propose a proactive online learning framework, called OCE-CRS (Fig. 1), that models the processing time estimation and scheduling task as a contextual combinatorial bandit problem [12], [13]. At the heart of our framework is an intelligent agent on the edge server that employs a novel algorithm named CN_DUCB. This agent learns to predict request durations based on the semantic content of the prompts. In each scheduling round, it prioritizes a batch of requests with the shortest estimated processing times for local execution on the edge, then offloads the remaining requests predicted to be more time-consuming to the cloud. This action serves a dual purpose: it not only maximizes edge-specific throughput [14], [15] but also implements a system-wide SJF policy that enhances overall internal efficiency by mitigating HOL blocking. A key feature of our design is its handling of delayed feedback; the actual processing time for a request is only known after it is complete. To prevent this delay from stalling the system, we design an asynchronous training strategy that allows the prediction model to be updated continuously without impeding real-time scheduling decisions [16]. This allows the system to effectively learn and adapt online to the combined effects of hardware, model, and workload. We provide a theoretical regret analysis for our algorithm and validate its efficacy through extensive experiments. Our evaluation confirms the framework's superiority in a standard static batching model and further demonstrates its robustness and adaptability in a simulated high-throughput environment mimicking modern continuous batching systems.

In this work, we address the complexity and unpredictability of scheduling dynamic LLM requests in cloud-edge environments. Our main contributions are:

- To the best of our knowledge, we are the first to introduce a framework that tackles throughput and responsiveness optimization for cloud-edge load balancing in LLMs by framing the processing time estimation problem as a contextual combinatorial bandit task.
- We design an innovative scheduling algorithm based on contextual combinatorial semi-bandits with delayed feedback, which serves as the core of our framework. It employs a neural UCB method to learn request durations and uses an asynchronous training mechanism to decouple time-consuming model updates from the real-time scheduling process, thereby maximizing edge throughput, offloading complex tasks, as well as ensuring practical real-time adaptability.
- We provide a theoretical analysis that demonstrates a sublinear regret bound for our algorithm under delayed feedback, thereby advancing the theoretical understanding of online learning for LLM scheduling.
- We validate our approach through extensive experiments on simulated and real-world testbeds, confirming its effectiveness in improving system throughput and responsiveness. Its superiority and robustness are demonstrated in both static and continuous batching environments, confirming its applicability to modern LLM serving systems.

Organization: Section II provides a summary of related work. The system model and the formulation are presented in Section III. Section IV details the proposed algorithm. The theoretical analysis is offered in Section V. Section VI contains the experimental comparisons and performance evaluation. Finally, the paper is concluded in Section VII.

II. Related Work

LLM Inference and Scheduling Systems: Optimizing LLM serving is an active research area. Systems like Orca [17] use fine-grained, iteration-level scheduling and selective batching to enhance fairness and throughput, while vLLM [18] addresses KV cache fragmentation with PagedAttention to support larger batches and better memory use. These systems are foundational for high-throughput inference, largely due to their implementation of continuous batching. However, while revolutionary in maximizing hardware utilization, their core scheduling logic often defaults to FCFS, making them prone to HOL blocking. To demonstrate that our scheduling approach is compatible with and effective in such modern high-performance settings, our performance evaluation is conducted not only in a basic static batching model but also in a simulated environment designed to functionally mirror the high-concurrency conditions of continuous batching.

More recent work tackles the HOL issue directly: FastServe [19] introduces token-level preemption with a Multi-Level Feedback Queue to interrupt long jobs, and Qiu et al. [20] use a proxy model to predict output length for SJF scheduling. While effective, these methods have limitations: preemption

is reactive and incurs overhead, while predictive SJF requires internal model access, which may not be feasible for black-box or fast-evolving LLMs. In contrast, our approach is proactive and model-agnostic: by casting scheduling as a contextual bandit problem, we learn to predict job durations from request content alone, enabling early short-job scheduling on the edge to prevent HOL blocking altogether.

Cloud-Edge Collaboration for AI/LLM: Offloading AI tasks from edge to cloud is a widely studied approach to balance latency and computational demand. The architecture for such collaboration is itself an active area of research, with recent works proposing multi-tier frameworks for hierarchical scheduling and resource management [21]. Within these systems, many studies leverage Deep Reinforcement Learning (DRL) for creating offloading policies [22], [23]. For instance, Wang et al. [24] use DRL to optimize LLM subtask offloading based on system states like network and server load, while He et al. [25] adopt active inference to address DRL's sample inefficiency. Similarly, in the context of federated learning, state-adaptive scheduling has been explored to manage resource competition on IoT devices at the edge [26]. These approaches focus on learning general offloading policies, often for tasks that are decomposable or have more predictable characteristics. In contrast, our framework aims to maximize edge throughput, with offloading emerging naturally from a content-aware scheduling mechanism: the bandit agent identifies long-running requests and redirects them to the cloud, enabling fine-grained, adaptive resource management beyond coarse system-state decisions.

Combinatorial Contextual Bandits: Our scheduling method builds upon advancements in combinatorial contextual bandits. These algorithms are designed for settings where an agent must repeatedly select a subset of available "arms" (options) based on contextual information to maximize a cumulative reward, often under uncertainty. Foundational work in this area applied the framework to recommendation systems, focusing on item diversity and achieving low regret [12], and later extended it to handle general nonlinear reward functions [27]. This powerful framework has proven particularly effective for resource management in modern distributed systems. For instance, in the domain of edge and cloud computing, Chen and Xu [28] utilized a contextual combinatorial bandit to optimize task replication in vehicular clouds, notably also addressing the challenge of delayed feedback. Similarly, Yang et al. [29] leveraged this paradigm to manage computation offloading in edge networks with unknown resource availability. These studies highlight the suitability of combinatorial bandits for dynamic decision-making at the network edge.

More recent advancements have incorporated deep learning, leading to neural bandits such as NeuralUCB [30] and Combinatorial Neural Bandits [31]. These methods leverage the power of neural networks to model complex relationships between context and rewards, aiming for efficient learning in high-dimensional context spaces. A key distinction of our work is the explicit treatment of delayed feedback within this more complex neural combinatorial setting. While delayed feedback has been studied in linear contextual bandits, including in the edge computing context [28], its analysis for combinatorial tasks using neural network predictors remains largely unexplored. Our CN_DUCB algorithm addresses this gap and, to our knowledge, offers the first theoretical regret analysis in this specific context.

III. Model and Formulation

This section details the system architecture and the problem formulation. We first introduce the cloud-edge collaborative architecture and the overall request processing workflow. Then, we describe the scheduling process and formulate the online decision-making problem as a contextual combinatorial semi-bandit task.

A. System Architecture

We consider a two-tier cloud-edge collaborative system designed to serve LLM inference requests from a set of end-users, as conceptually illustrated in the overall framework diagram (Fig. 1). The system is composed of an edge tier and a cloud tier, whose roles and characteristics are defined as follows.

- *Edge Tier:* This tier consists of a set of geographically distributed edge servers. Each edge server is located in proximity to end-users and is equipped with computational resources (e.g., GPUs) for LLM inference. The primary advantage of the edge tier is its ability to provide low-latency responses due to its closeness to users. However, its computational and memory resources are constrained compared to the cloud. A key component of our framework, an intelligent scheduling agent, is hosted on each edge server.
- *Cloud Tier:* This tier consists of a powerful, centralized cloud data center. It possesses vast, scalable computational resources, making it suitable for handling computationally intensive, long-running LLM requests. The main drawback of the cloud is the significant network latency incurred due to the round-trip time (RTT) between the edge and the cloud.

The request workflow within this architecture proceeds as follows. An LLM request from a user is first routed to its nearest edge server. The scheduling agent on that server intercepts the request and places it into a local waiting queue. The agent's core responsibility is to make a real-time decision for each batch of requests: either (a) schedule them for local execution on the edge server's resources, or (b) offload them to the cloud data center for processing. This strategic decision-making process is the central focus of our work.

B. Scheduling Process and Bandit Formulation

Our proposed scheduling system dynamically sorts incoming LLM requests based on their estimated processing times. This strategy is designed to achieve a dual objective. First, by dedicating the resource-constrained edge to faster tasks, it maximizes the edge server's throughput, consequently boosting the overall system's request processing capacity. Second, the SJF-based approach mitigates HOL blocking across the system, which enhances internal scheduling efficiency and leads to significant improvements in user-centric metrics like average Job Completion Time (JCT).

To formalize this online decision-making process under uncertainty, we model it as a contextual combinatorial semi-bandit problem. Each incoming LLM request is treated as an arm. Each request is encoded into an embedding $\boldsymbol{x} \in \mathbb{R}^d$ by an encoder, which serves as input to a neural network $f(\boldsymbol{x}; \boldsymbol{\theta})$ with parameter $\boldsymbol{\theta}$ to predict its utility. We define the time window during which an edge server processes a batch of requests as a round, indexed

by t. The batch size is denoted as K_t. We consider a high-concurrency scenario where the number of queued requests, N_t, satisfies $N_t \geq K_t$ for all time. Let T denote the time horizon of request processing.

Let $\mathcal{S}_t \subseteq 2^{[N_t]}$ be the set of all allowed subsets of queuing requests at the beginning of round t. The agent selects a subset of requests $S_t \in \mathcal{S}_t$ with the constraint $|S_t| = K_t$ to be processed in the current batch on the edge server. K_t is determined by the capacity of the edge server, and we assume there exists an upper bound K such that $K_t \leq K$ for any $t \in [T]$. The remaining requests in the queue are offloaded to the cloud. Unlike traditional bandits, our semi-bandit setting reveals the feedback for all arms within the chosen subset S_t, meaning the actual processing duration is observed for every request selected for local execution.

C. Utility and Reward Functions

Since our goal is to enhance the throughput of the edge server, which is equivalent to reducing the average processing time, requests with shorter execution times are preferred for local processing. Hence, the utility $u_{t,i}$ of arm i at round t is defined as the negative processing time of this LLM request. We denote the processing time as $\psi_{t,i}$, then $u_{t,i} = -\psi_{t,i}$. The processing time $\psi_{t,i}$ is influenced by the output token length of the request, as described by $\psi_{t,i} = p \cdot n_{t,i} + c$, where p is the latency per token, $n_{t,i}$ is the number of tokens in request i, and c is the system's overhead [20]. As the output token length $n_{t,i}$ is unknown in advance, the true utility $u_{t,i}$ is unknown a priori and is only observed after the request is processed. We model the utility as

$$u_{t,i} = h(\boldsymbol{x}_{t,i}) + \epsilon_t,$$

where $h(\cdot)$ is an unknown function and ϵ_t is a ν-sub-Gaussian noise conditioned on the history $\mathcal{F}_t = \{S_s \mid s < t\}$ with $\mathbb{E}[\epsilon_t \mid \mathcal{F}_t] = 0$. For a selected batch (super-arm) S_t, the reward $R(S_t, \boldsymbol{u}_t)$ is defined as the minimum utility among the arms in S_t,

$$R(S_t, \boldsymbol{u}_t) = \min_{i \in S_t} u_{t,i}.$$

Since utility is the negative of processing time, this corresponds to the negative of the longest processing time within the batch. This models the behavior of static batching, where the completion time of a batch is dictated by its slowest task.

D. Delayed Feedback and Optimization

To maintain real-time processing efficiency, we employ asynchronous training for the neural network used to estimate utilities. Feedback generated during round t may not be used immediately. Instead, it is accumulated and incorporated into model updates at the end of subsequent rounds. This results in delayed feedback, where the rewards from one round may influence the model's training in later rounds. Specifically, for any $i \in S_t$, the true utility $u_{t,i}$ is observed by the model after $d_{t,i}$ rounds. At round t, the agent receives a feedback set $\{u_{s,i} | (s,i) \in \mathcal{T}_t^r\}$ where $\mathcal{T}_t^r = \{(s,i) | i \in S_s, s + d_{s,i} = t\}$. Let $\mathcal{T}_t$ denote the set of all received feedback indexes up to round t, i.e., $\mathcal{T}_t = \cup_{s=1}^{t} \mathcal{T}_s^r$.

The optimization goal is to maximize the throughput of the edge server, which is equivalent to minimizing the cumulative expected regret $\mathcal{R}(T)$. This regret quantifies the loss in potential rewards due to suboptimal decision-making over T rounds.

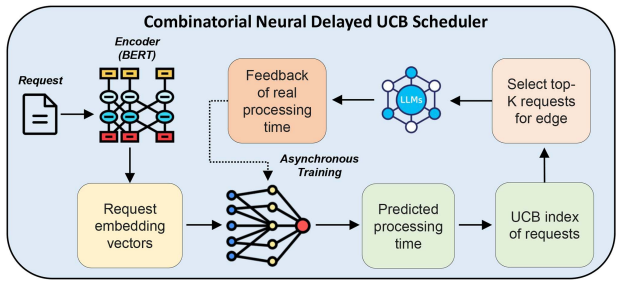


Fig. 2. The workflow of our online request selection algorithm, CN_DUCB. Neural network parameters are optimized through gradient descent. To maintain real-time processing efficiency, the scheduling process and the training process are asynchronous, resulting in delayed feedback.

Formally, the regret $\mathcal{R}(T)$ is defined as

$$\mathcal{R}(T) = \mathbb{E}\left[\sum_{t=1}^{T} \left(R(S_t^*, \boldsymbol{u}_t) - R(S_t, \boldsymbol{u}_t)\right)\right],$$

where $\boldsymbol{u}_t = \{u_{t,i}\}_{i=1}^{N_t}$ is the utility vector and $S_t^* = \arg\max_{S \in \mathcal{S}_t} R(S, \boldsymbol{u}_t)$ is the offline optimal super arm at round t.

IV. Algorithm Design

Due to the uncertainties introduced by the types of LLMs, their output generation mechanisms, and deployment environments, it is challenging to predict the processing time for different LLM requests. As a result, existing LLM serving systems primarily rely on FCFS scheduling, which often leads to severe HOL blocking and degrades performance. To address this, we propose an Online Cloud-Edge Collaborative Request Scheduling (OCE-CRS) framework. We leverage the powerful learning capabilities of neural networks to continuously explore and exploit the intrinsic relationship between LLM requests and their processing times. This enables a dynamic, SJF-inspired policy that simultaneously enhances system throughput and overall scheduling efficiency online.

To implement this SJF-based policy online, the agent must repeatedly select a subset of requests that are predicted to be the fastest, all while facing uncertainty about their true processing times. This sequential decision-making process, where the goal is to maximize cumulative rewards (i.e., minimize processing times) based on contextual information from each request, can be naturally formulated as a contextual combinatorial bandit problem. To solve this, we propose a Combinatorial Neural Delayed Upper Confidence Bound (CN_DUCB) scheduling algorithm. The core idea of CN_DUCB is to use a neural network to predict the processing time of each request and then calculate a score based on the Upper Confidence Bound (UCB) principle, which balances exploiting requests with known short execution times and exploring uncertain ones. The requests with the highest scores (indicating a higher probability of being "short jobs") are then selected for execution. Furthermore, to handle the inherent delay between making a decision and observing its outcome (the actual processing time), the algorithm incorporates an asynchronous training mechanism, ensuring that model updates do not compromise the system's real-time scheduling performance. The overall workflow is illustrated in Fig. 2.

In our proposed method, BERT-base is used as a proxy model to encode arriving requests by taking the classification token (CLS) as the input context vector of the following approximation

model [32]. We utilize a neural network function $f(\boldsymbol{x};\boldsymbol{\theta})$ with a fully connected architecture to approximate $h(\cdot)$, as defined by

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \sqrt{m}\boldsymbol{W}_L\sigma\left(\boldsymbol{W}_{L-1}\sigma\left(\cdots\sigma\left(\boldsymbol{W}_1\boldsymbol{x}\right)\right)\right),$$

where $\sigma(\cdot)$ is the ReLU activation function, and $\boldsymbol{W}_i$ are the weight matrices [30]. Without loss of generality, we assume that the width of each hidden layer is the same (i.e., m) for convenience in analysis. The parameter matrix can be represented as $\boldsymbol{\theta} = [\text{vec}(\boldsymbol{W}_1)^\top, \ldots, \text{vec}(\boldsymbol{W}_L)^\top]^\top \in \mathbb{R}^p$ with $p = (1+d)m + (L-1)m^2$, where $\text{vec}(\cdot)$ vectorizes a matrix into a flatten vector. The gradient of this function is denoted by $\boldsymbol{g}(\boldsymbol{x};\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{x};\boldsymbol{\theta})$. Given the confidence parameter $\delta \in (0,1)$, we define the scaling factor as

$$\gamma_t = \Lambda_{1,t}\left(\nu\sqrt{\log\frac{\det \boldsymbol{Z}_t}{\det \lambda\boldsymbol{I}} + \Lambda_{2,t}} + S\sqrt{\lambda}\right) + \Lambda_{3,t},$$

where $\Lambda_{1,t} = (1+\bar{B}_1 t^{\frac{7}{6}} K^{\frac{7}{6}} L^4 \lambda^{-\frac{7}{6}} m^{-\frac{1}{6}}\sqrt{\log m} + \bar{B}_2 d_m KL\lambda^{-1})^{\frac{1}{2}}$, $\Lambda_{2,t} = \bar{B}_3 t^{\frac{5}{3}} K^{\frac{5}{3}} L^4 \lambda^{-\frac{7}{6}} m^{-\frac{1}{6}}\sqrt{\log m} + \bar{B}_4 d_m^{\frac{3}{2}} K^{\frac{3}{2}} L\lambda^{-1} - 2\log\delta$, and $\Lambda_{3,t} = (\lambda + \bar{B}_5 tKL)((1-\eta m\lambda)^{\frac{J}{2}}\sqrt{tK/\lambda} + t^{\frac{5}{3}} K^{\frac{5}{3}} L^{\frac{7}{2}} \lambda^{-\frac{5}{3}} m^{-\frac{1}{3}}\sqrt{\log m}(1+\sqrt{tK/\lambda}))$ for some positive constants $\{\bar{B}_i\}_{i=1}^5$. This scaling factor is a mechanism that balances exploration and exploitation by adjusting the confidence intervals of the predictions and is derived from the theoretical analysis in the next section. The parameters of the network are initialized from a Gaussian distribution. Specifically, for $1 \le l \le L-1$, assign $\boldsymbol{W}_l = (\boldsymbol{W}, \boldsymbol{0}; \boldsymbol{0}, \boldsymbol{W})$, where each $\boldsymbol{W}$ generated independently from $\mathcal{N}(0, 2/m)$.

The requests in the queue are first be sorted in descending order according to a score $U_{t,i} + \omega_t$, where $U_{t,i}$ is the confidence interval and ω_t is an offset, defined respectively as follows.

$$U_{t,i} = \hat{u}_{t,i} + \gamma_{t-1}\|\boldsymbol{g}(\boldsymbol{x}_{t,i};\boldsymbol{\theta}_{t-1})\|_{\boldsymbol{Z}_{t-1}^{-1}}/\sqrt{m}, \tag{1}$$

$$\omega_t = \bar{B}_1\gamma_{t-1} t^{\frac{1}{6}} K^{\frac{1}{6}} L^{\frac{7}{2}} \lambda^{-\frac{2}{3}} m^{-\frac{1}{6}}\sqrt{\log m} + \bar{B}_2 t^{\frac{2}{3}} K^{\frac{2}{3}} L^3 \lambda^{-\frac{2}{3}} m^{-\frac{1}{6}}\sqrt{\log m}. \tag{2}$$

Following this SJF-based principle, we greedily select the top-K_t requests as a batch to be executed locally on the edge and offload the remaining to the cloud. The execution duration of each request on the edge is recorded for training $\boldsymbol{\theta}$. Once the previous training round is completed, the new data can be fed into the model to begin the next round. The detailed scheduling procedure is illustrated in Algorithm 1, where the scheduling process and the training process are asynchronous, ensuring that the relatively time-consuming model training does not affect the real-time performance of the scheduling. The *trainSignal* in Algorithm 1 serves as an abstraction to illustrate the asynchronous nature of our framework. In practice, this is implemented with a dedicated training thread that runs decoupled from the real-time scheduling loop. The training process is event-driven: a new training cycle is initiated whenever the training thread is idle and a sufficient batch of new feedback (i.e., observed processing times from completed requests) has been collected. This design ensures that the time-consuming model training process does not introduce latency into the critical path of scheduling incoming requests, thereby maintaining the system's responsiveness.

During training, the model parameters $\boldsymbol{\theta}$ are optimized through gradient descent. After training is complete, the updated parameters are synchronized with the inference model. The inference and model training approaches are summarized in Algorithm 2 and Algorithm 3, respectively. Note that model training may span multiple scheduling rounds, during which feedback is collected and then fed into the model once the current training round is complete. We model this process using bandits with delayed feedback and analyze our algorithm's performance in the next section, resorting to the bandit theory framework.

Algorithm 1: Cloud-Edge Scheduling.

input : LLM request sequence $\boldsymbol{q}_t$, Number of rounds T, *trainSignal*

Initialize $\boldsymbol{\theta}_0$ and set $\boldsymbol{Z}_0 = \lambda\boldsymbol{I}$;
for $t \in \{1, 2, \cdots, T\}$ **do**
  Receive LLM requests $\{q_{t,i}\}_{i\in[N_t]}$ in queue;
  Compute $\boldsymbol{U}_t, \omega_t = \text{UCBInference}(\boldsymbol{q}_t, \boldsymbol{\theta}_{t-1}, \boldsymbol{Z}_{t-1})$;
  Sort queue in descending order based on $U_{t,i} + \omega_t$;
  Choose $\boldsymbol{S}_t$ by picking top-K_t requests for the edge and offload the remaining to the cloud;
  Processing requests in batch and collect feedback $\{\psi_{t,i}\}_{i\in\boldsymbol{S}_t}$;
  if *trainSignal* = ***True*** **then**
    $\boldsymbol{\theta}_t, \boldsymbol{Z}_t = \text{TrainingNN}(\eta, J, m, \boldsymbol{\theta}_{t-1})$;
  else
    Assign $\boldsymbol{Z}_t = \boldsymbol{Z}_{t-1}$, $\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1}$;

Algorithm 2: UCBInference.

input : request sequence $\boldsymbol{q}_t$, $\boldsymbol{\theta}_{t-1}$, $\boldsymbol{Z}_{t-1}$, BertBase() model, regularization parameter λ, network width m

for $q_{t,i} \in \{q_{t,1}, q_{t,2}, \cdots, q_{t,N_t}\}$ **do**
  Encode request prompts as $\boldsymbol{x}'_{t,i} = \text{BertBase}(q_{t,i})$;
  Define $\boldsymbol{x}_{t,i} = [\boldsymbol{x}'^\top_{t,i}, \boldsymbol{x}'^\top_{t,i}]^\top$;
  Compute $\hat{u}_{t,i} = f(\boldsymbol{x}_{t,i};\boldsymbol{\theta}_{t-1})$ and $U_{t,i} = \hat{u}_{t,i} + \gamma_{t-1}\|\boldsymbol{g}(\boldsymbol{x}_{t,i};\boldsymbol{\theta}_{t-1})\|_{\boldsymbol{Z}_{t-1}^{-1}}/\sqrt{m}$;
Compute ω_t;
return $\boldsymbol{U}_t$, ω_t

Algorithm 3: TrainingNN.

input : Regularization parameter λ, step size η, number of gradient descent steps J, network width m, contexts $\{\boldsymbol{x}_{s,j}\}_{(s,j)\in\mathcal{T}_t}$, utility $\{u_{s,j}\}_{(s,j)\in\mathcal{T}_t}$, initial parameter $\boldsymbol{\theta}^{(0)}$.

Define $\mathcal{L}(\boldsymbol{\theta}) = \sum_{k=1}^{t}\sum_{i\in S_k}(f(\boldsymbol{x}_{k,i};\boldsymbol{\theta}) - u_{k,i})^2/2 + m\lambda\|\boldsymbol{\theta} - \boldsymbol{\theta}_0\|_2^2/2$;
for $j = 0, \ldots, J-1$ **do**
  $\boldsymbol{\theta}^{(j+1)} = \boldsymbol{\theta}^{(j)} - \eta\nabla\mathcal{L}(\boldsymbol{\theta}^{(j)})$;
Update $\boldsymbol{\theta}_t = \boldsymbol{\theta}^{(J)}$ and $\boldsymbol{Z}_t = \boldsymbol{Z}_{t-1} + \sum_{(s,j)\in\mathcal{T}_t^r}\boldsymbol{g}(\boldsymbol{x}_{s,i};\boldsymbol{\theta}_{s-1})\boldsymbol{g}(\boldsymbol{x}_{s,i};\boldsymbol{\theta}_{s-1})^\top/m$;
Return $\boldsymbol{\theta}_t$, $\boldsymbol{Z}_t$

V. Theoretical Analysis

In this section, we provide a rigorous theoretical analysis for our proposed CN_DUCB algorithm. We begin by presenting our main theoretical result, Theorem 1, which establishes a sublinear regret bound for the algorithm under delayed feedback. The

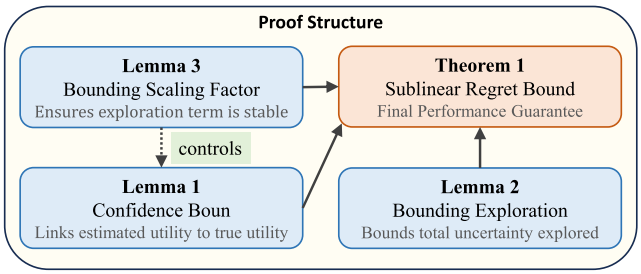


Fig. 3. Logical structure of the regret analysis. The diagram illustrates how the main result, Theorem 1, is derived from three key supporting lemmas that establish confidence bounds (Lemma 1), bound the cumulative exploration (Lemma 2), and ensure stability (Lemma 3).

proof of this theorem is non-trivial and relies on a sequence of key supporting lemmas. Specifically, our analysis proceeds as follows. First, we establish high-probability confidence bounds that relate the neural network's estimated utility to the true (but unknown) utility function (Lemma 1). Second, we bound the cumulative exploration term across all rounds by analyzing the sum of squared, weighted gradient norms, which represents the total confidence width explored by the policy (Lemma 2). Third, we prove that the exploration scaling factor, γ_t, remains well-behaved and bounded throughout the learning process, which is crucial for ensuring the stability of our confidence bounds (Lemma 3). Finally, we combine these components to formally derive the overall cumulative regret in the proof of Theorem 1. The logical flow of this analysis is visually summarized in Fig. 3.

For simplicity, we assume the utility $\boldsymbol{u}$ can be characterized by $h(\boldsymbol{x}) + \epsilon_t$ and $\boldsymbol{u}$, scaled with $h(\boldsymbol{x}) \in [0, 1]$. In our problem, the reward function is specifically defined as $R(S_t, \boldsymbol{u}_t) = \min_{i \in S_t} u_{t,i}$, representing the execution duration for a batch of requests. Actually, our theoretical results can be extended to any reward function that satisfies monotonicity and Lipschitz continuity.

A. Regret of CN_DUCB

We start with two definitions of neural tangent kernel theory [33] based on which our analysis is built.

Definition 1: Let $\{x_{t,i} \mid t \in [T], i \in [N]\}$ be a set of contexts. Define

$$\widetilde{\boldsymbol{H}}_{i,j}^{(1)} = \boldsymbol{\Sigma}_{i,j}^{(1)} = \langle \boldsymbol{x}^i, \boldsymbol{x}^j \rangle, \quad \boldsymbol{A}_{i,j}^{(l)} = \begin{pmatrix} \boldsymbol{\Sigma}_{i,i}^{(l)} & \boldsymbol{\Sigma}_{i,j}^{(l)} \\ \boldsymbol{\Sigma}_{i,j}^{(l)} & \boldsymbol{\Sigma}_{j,j}^{(l)} \end{pmatrix},$$

$$\boldsymbol{\Sigma}_{i,j}^{(l+1)} = 2\mathbb{E}_{(u,v)\sim N(\boldsymbol{0}, \boldsymbol{A}_{i,j}^{(l)})} \left[\sigma(u)\sigma(v)\right],$$

$$\widetilde{\boldsymbol{H}}_{i,j}^{(l+1)} = 2\widetilde{\boldsymbol{H}}_{i,j}^{(l)} \mathbb{E}_{(u,v)\sim N(\boldsymbol{0}, \boldsymbol{A}_{i,j}^{(l)})} \left[\sigma'(u)\sigma'(v)\right] + \boldsymbol{\Sigma}_{i,j}^{(l+1)}.$$

Then, $\boldsymbol{H} = (\widetilde{\boldsymbol{H}}^{(L)} + \boldsymbol{\Sigma}(L))/2$ is called the neural tangent kernel (NTK) matrix on the context set [33], [34].

Definition 2: The effective dimension $\widetilde{d}$ of the NTK matrix $\boldsymbol{H}$ with regularization parameter λ is defined as

$$\widetilde{d} = \log\det(\boldsymbol{I} + \boldsymbol{H}/\lambda)/\log(1 + TN/\lambda).$$

We also assume $\boldsymbol{H} \succeq \lambda_0 \boldsymbol{I}$ (i.e. non-singular NTK matrix) and $\|\boldsymbol{x}_{t,i}\|_2 = 1$, which are mild and commonly used in the literature of neural contextual bandits. The core element of our proposed scheduling framework is called CN_DUCB algorithm. The following theorem presents the regret upper bound of CN_DUCB, as our main theoretical result.

Theorem 1: There exist positive constants $\{B_i\}_{i=1}^6$ and denote by $\beta = \lfloor (T - d_m - 1)/d_m \rfloor$ and $\omega_t = B_1 \gamma_{t-1} t^{\frac{1}{6}} K^{\frac{1}{6}} L^{\frac{7}{2}} \lambda^{-\frac{2}{3}} m^{-\frac{1}{6}} \sqrt{\log m} + B_2 t^{\frac{2}{3}} K^{\frac{2}{3}} L^3 \lambda^{-\frac{2}{3}} m^{-\frac{1}{6}} \sqrt{\log m}$. For and any $\delta \in (0, 1)$, if $m \geq poly(T, L, N, \lambda^{-1}, \lambda_0^{-1}, \log T)$, $\eta = B_3(TKmL + m\lambda)^{-1}$, $\lambda \geq B_4 LK$, $J = 2\log(\lambda S/(\sqrt{T}(\lambda + B_5 TL)))TKL/\lambda = \widetilde{\mathcal{O}}(TL/\lambda)$, $\lambda \geq \max(1, S^{-2})$, and $S \geq \sqrt{2\boldsymbol{h}^\top \boldsymbol{H}^{-1}\boldsymbol{h}}$, then with probability at least $1 - \delta$, the cumulative expected regret of the proposed algorithm satisfies

$$\begin{aligned} \mathcal{R}(T) \leq\ & (8(T - d_m)Kd_m(\widetilde{d}\log(1 + TN/(\lambda d_m)) + 3))^{\frac{1}{2}} \\ & \cdot (\nu((1 + B_4 d_m KL\lambda^{-1})(\widetilde{d}\log(1 + TN/\lambda) + 1 \\ & + 2B_6 d_m^{\frac{3}{2}} K^{\frac{3}{2}} L\lambda^{-1} - 2\log\delta))^{\frac{1}{2}} \\ & + S\sqrt{\lambda + B_4 d_m KL} + 2) + 2Kd_m + 2, \end{aligned}$$

where ν and λ treated as constants, d_m is the maximum number of delayed rounds, and $S = \sqrt{2\boldsymbol{h}^\top \boldsymbol{H}^{-1}\boldsymbol{h}}$. The regret bound becomes $\mathcal{R}(T) = \widetilde{\mathcal{O}}(\sqrt{\widetilde{d}TKd_m \max\{\widetilde{d}, K\}})$.

Remark 1: Theorem 1 provides a theoretical guarantee for the performance of the scheduling algorithm by establishing a sublinear regret bound. This bound indicates that, with special training iteration number J in Algorithm 3, as the number of scheduling rounds increases, the cumulative regret grows at a much slower rate compared to the number of rounds. Intuitively, this becomes increasingly effective over time in selecting the best requests for edge processing.

B. Proof of Theorem 1

Now, we present a complete proof for Theorem 1. Before analyzing the regret, we first introduce several supporting lemmas. To avoid making stringent parametric assumptions, we utilize overparameterized neural networks, which lead to a large hidden layer size m, and consequently p. Therefore, ensuring that the regret bound remains independent of m is essential. Lemma 1 demonstrates that the upper confidence bound $u_{t,i}$ remains close to the expected score $U_{t,i}$ and provides the exact value of the offset term.

Lemma 1: There exist positive constant $\bar{B}_1, \bar{B}_2$ such that for any $\delta \in (0, 1)$, if $\eta \leq \bar{B}_3(TKmL + m\lambda)^{-1}$ for some positive $\bar{B}_3$, then with probability at least $1 - \delta$, we have

$$\mathbb{E}\left[|U_{t,i} - u_{t,i}|\right] \leq 2\gamma_t \|\boldsymbol{g}(\boldsymbol{x}_{t,i}; \boldsymbol{\theta}_{t-1})/\sqrt{m}\|_{\boldsymbol{Z}_{t-1}^{-1}} + \omega_t,$$

where $\omega_t = \bar{B}_1 \gamma_{t-1} t^{\frac{1}{6}} K^{\frac{1}{6}} L^{\frac{7}{2}} \lambda^{-\frac{2}{3}} m^{-\frac{1}{6}} \sqrt{\log m} + \bar{B}_2 t^{\frac{2}{3}} K^{\frac{2}{3}} L^3 \lambda^{-\frac{2}{3}} m^{-\frac{1}{6}} \sqrt{\log m}$. Furthermore, we have $\mathbb{E}[U_{t,i} + \omega_t] \geq \mathbb{E}[u_{t,i}]$.

Unlike the case where only one is chosen each time in [30], the combinatorial setting makes it harder to upper bound the reward of a subset of arms. We follow the idea in [31] to introduce an offset ω_t, then prove Lemma 1 under the condition of delayed feedback. The employment of Lemma 1 in regret analysis would introduce γ_T and the sum of weighted norms, both of which are unbounded yet. We further present Lemma 2 and Lemma 3 to address these challenges, Lemma 2 for the sum of weighted norms and Lemma 3 for γ_T. Although there are similar trials of Lemma 2 in literature [12], [30], [31], we derive an upper bound

in the context of combinatorial neural bandits with delayed feedback for the first time. Due to the space constraint, the proofs of Lemma 1−3 [30], [31], [34], [35], [36], [37], [38] are relegated to Appendix A, available online.

Lemma 2: For any $\delta \in (0,1)$ and some positive constant $\bar{B}_1, \bar{B}_2, \bar{B}_3$ where $\bar{B}_2 \geq \sqrt{\max_{t,i} \|\boldsymbol{g}(\boldsymbol{x}_{t,i};\boldsymbol{\theta}_{t-1})/\sqrt{m}\|_2^2/L}$, if $\lambda \geq \bar{B}_2 LK$, $\eta \leq \bar{B}_1(TKLm+\lambda m)^{-1}$, $m \geq \bar{B}_2 K^{-\frac{1}{2}} L^{-\frac{3}{2}} \lambda^{\frac{1}{2}} (\log(TNL^2/\delta))^{\frac{3}{2}}$, and $m(\log m)^{-3} \geq \bar{B}_2(TKL^{12} \lambda^{-1} + T^4K^4L^18\lambda^{-10}(\lambda + TKL)^6 + T^7K^7L^{21}\lambda^{-7}(1+\sqrt{TK/\lambda})^6)$, then with probability at least $1-\delta$, we have

$$\sum_{t=d_m+1}^{(\beta+1)d_m+1} \sum_{i\in S_t} \|\boldsymbol{g}(\boldsymbol{x}_{t,i};\boldsymbol{\theta}_{t-1})/\sqrt{m}\|^2_{\boldsymbol{Z}_{t-1}^{-1}} \leq d_m \cdot \min\Big\{\beta, 2\widetilde{d}\log(1+\beta N/\lambda)+2 + \bar{B}_3\beta^{\frac{5}{3}}K^{\frac{5}{3}}L^4\lambda^{-\frac{1}{6}}m^{-\frac{1}{6}}\sqrt{\log m}\Big\}.$$

Lemma 3: For some positive constants $\{\bar{B}_i\}_{i=1}^4$, and any $\delta \in (0,1)$, if $m \geq poly(T, L, N, \lambda^{-1}, \lambda_0^{-1}, \log T)$, $J = 2\log(\lambda S/(\sqrt{T}(\lambda + \bar{B}_3 TL)))TKL/\lambda = \widetilde{\mathcal{O}}(TL/\lambda)$, then with probability at least $1-\delta$, we can bound the scaling factor γ_T as follows.

$$\begin{aligned}\gamma_T \leq\ & (1 + \bar{B}_1 t^{\frac{7}{6}} K^{\frac{7}{6}} L^4 \lambda^{-\frac{7}{6}} m^{-\frac{1}{6}} \sqrt{\log m} + \bar{B}_2 d_m KL\lambda^{-1})^{\frac{1}{2}} \\ & \cdot (\nu(\widetilde{d}\log(1+TN/\lambda)+1 \\ & + 2\bar{B}_3 t^{\frac{5}{3}} K^{\frac{5}{3}} L^4 \lambda^{-\frac{7}{6}} m^{-\frac{1}{6}} \sqrt{\log m} + 2\bar{B}_4 d_m^{\frac{3}{2}} K^{\frac{3}{2}} L\lambda^{-1} \\ & - 2\log\delta)^{\frac{1}{2}} + S\sqrt{\lambda}) \\ & + (\lambda + \bar{B}_3 TKL)((1-\eta m\lambda)^{\frac{J}{2}}\sqrt{TK/\lambda} \\ & + T^{\frac{5}{3}} K^{\frac{5}{3}} L^{\frac{7}{2}} \lambda^{-\frac{5}{3}} m^{-\frac{1}{6}} \sqrt{\log m}(1+\sqrt{TK/\lambda})).\end{aligned}$$

Based on the above-mentioned Lemma 1−3, we can directly derive Theorem 1 as follows.

Proof of Theorem 1: $\mathcal{R}(T) =$

$$\begin{aligned}&= \mathbb{E}\left[\sum_{t=1}^{T}(R(S_t^*, \boldsymbol{u}_t) - R(S_t, \boldsymbol{u}_t))\right] \\ &\leq \sum_{t=d_m+1}^{(\beta+1)d_m+1} \mathbb{E}[R(S_t^*, \boldsymbol{u}_t) - R(S_t, \boldsymbol{u}_t)] + 2d_m K \\ &\overset{a}{\leq} \sum_{t=d_m+1}^{(\beta+1)d_m+1} \mathbb{E}[R(S_t^*, \boldsymbol{U}_t + \omega_t) - R(S_t, \boldsymbol{u}_t)] + 2d_m K \\ &\overset{b}{\leq} \sum_{t=d_m+1}^{(\beta+1)d_m+1} \mathbb{E}[R(S_t, \boldsymbol{U}_t + \omega_t) - R(S_t, \boldsymbol{u}_t)] + 2d_m K \\ &\leq \sum_{t=d_m+1}^{(\beta+1)d_m+1} \sum_{i\in S_t} \mathbb{E}[U_t + \omega_t - u_t] + 2d_m K \\ &\overset{c}{\leq} 2\sum_{t=d_m+1}^{(\beta+1)d_m+1} \sum_{i\in S_t} (\gamma_t \|\boldsymbol{g}(\boldsymbol{x}_{t,i};\boldsymbol{\theta}_{t-1})/\sqrt{m}\|_{\boldsymbol{Z}_{t-1}^{-1}}) \\ &\quad + 2TK\omega_T + 2d_m K \\ &\overset{d}{\leq} 2((T-d_m)K\gamma_T^2 \cdot d_m \cdot \min\{\beta, 2\widetilde{d}\log(1+\beta N/\lambda) \\ &\quad + 2 + \bar{B}_3\beta^{\frac{5}{3}}K^{\frac{5}{3}}L^4\lambda^{-\frac{1}{6}}m^{-\frac{1}{6}}\sqrt{\log m}\})^{1/2} \\ &\quad + 2TK\omega_T + 2d_m K,\end{aligned}$$

where Ineq. (a) follows from Lemma 1 and the monotonicity of our reward function $R(S_t, \cdot)$, Ineq. (b) follows from the fact that S_t is the optimal solution with respect to $\boldsymbol{U}_t + \omega_t$, Ineq. (c) follows from Lemma 1, and Ineq. (d) follows from Lemma 2. According to Lemma 3, γ_T can be scaled up with its upper bound. After substituting the upper bound of γ_T into the above inequality, with sufficiently large m, we can obtain the final result. □

TABLE I
VALIDATION OF MAE FOR DIFFERENT MLP ARCHITECTURES USED IN PROCESSING TIME PREDICTION

Layers	Width	MAE		Avg. MAE
		Phi-3	Mistral	
2	512	2.4610	2.3208	2.3909
3	512	2.4668	2.3176	2.3922
3	256	2.4662	2.3185	2.3924
2	**256**	**2.4634**	**2.3235**	**2.3934**
1	512	2.4754	2.3268	2.401
2	128	2.4826	2.3284	2.4055
3	128	2.4824	2.3353	2.4088
1	128	2.4781	2.3436	2.4109
1	256	2.4828	2.3324	2.4076

VI. PERFORMANCE EVALUATION

In this section, we first develop a simulator to evaluate the effectiveness, and then implement a testbed based on real-world processing times of LLM requests. Note that the calculation of $U_{t,i}$ in Algorithm 2 requires inverting the matrix $\boldsymbol{Z}_{t-1}$, which is usually computationally prohibitive. To manage computational efficiency, we adopt a standard approach from neural bandits, similar to methods used in NeuralUCB [30]. Specifically, we utilize a diagonal approximation of $\boldsymbol{Z}_{t-1}$ by retaining only its diagonal elements and setting all other matrix elements to zero. This avoids the high computational expense of matrix inversion.

A. Architecture of Prediction Model

To accurately estimate the unpredictable processing times of LLM requests, a critical component of our OCE-CRS framework is the prediction model. The inherent non-linearity in LLM request processing times necessitates a sophisticated prediction model beyond simpler linear approaches. We conduct a systematic architecture search for our Multi-Layer Perceptron (MLP) prediction model, which takes BERT embeddings of LLM request prompts as input. A BERT-base model provides 768-dimensional embeddings [32]. We explore MLP configurations varying the number of hidden layers ($L \in \{1,2,3\}$) and hidden layer width ($m \in \{128, 256, 512\}$). Rectified Linear Unit (ReLU) served as the activation function. Evaluation is performed on real-world LLM request inference data from the Alpaca dataset [39], specifically for the Phi-3-mini-4k-instruct [40] and Mistral-7B-Instruct-v0.3 [41] LLMs. Mean Absolute Error (MAE) was the primary evaluation metric. Models are trained with an Adam optimizer and MAE loss, utilizing early stopping (patience of 10 epochs). Table I presents the validation MAE for different MLP architectures across two LLMs.

Table I indicates that deeper and wider architectures generally yield lower MAE, signifying improved prediction accuracy. Single-layer models consistently showed higher MAE across all widths, supporting the need for non-linear models

to capture the complex relationship between prompt features and LLM processing times. This performance degradation for shallower/narrower networks confirms the non-linear nature of the problem. Based on these results, we select the fully connected MLP architecture with $L = 2$ and $m = 256$. Table I indicates that a 2-layer MLP is sufficient to capture the problem's complex non-linearity, as deeper or wider models (e.g., $L = 3$ or $m = 512$) offer only marginal improvements in MAE while incurring greater computational overhead. Therefore, our chosen configuration represents the most efficient architecture that provides top-tier prediction accuracy, avoiding unnecessary complexity for negligible gains.

B. Performance of Online Selection Algorithm

To validate the core component of our framework, this subsection evaluates the performance of the proposed online selection algorithm, CN_DUCB, before assessing it within the complete cloud-edge collaborative system in the subsequent section. The experiments aim to demonstrate the effectiveness of CN_DUCB by estimating LLM request processing times under various conditions, including non-linear reward functions and realistic delayed feedback. We compare its performance against established baselines in both simulated and real-world environments, highlighting its ability to balance exploration and exploitation efficiently.

1) Experimental Setup: Our experimental evaluation is conducted in both environments with both simulated reward functions and a real-world testbed.

For the simulated-reward experiments, we configure a contextual combinatorial bandit environment with a context dimension $d = 40$. The number of contextual arms per round is set to $N = 20$, from which $K = 4$ arms are selected for processing, mirroring a top-K selection problem. The rewards of arms are characterized by a reward function $h(\cdot)$ plus a noise ϵ where $\epsilon \sim \mathcal{N}(0, 1)$. We select two distinct non-linear reward functions to assess the algorithm's ability to handle complex relationships: $h_1(\boldsymbol{x}) = 10(\boldsymbol{x}^\top \boldsymbol{a})^2$ and $h_2(\boldsymbol{x}) = \boldsymbol{x}^\top A^\top A\boldsymbol{x}$. Here, $A \in \mathbb{R}^{d\times d}$ is randomly generated from a uniform distribution $\mathcal{U}[0, 1]$, and $\boldsymbol{a}$ is randomly generated from a unit ball [30]. These functions amplify the sensitivity of the reward to correct arm selection. The input context $\boldsymbol{x}$ for the bandit algorithms is derived from BERT embeddings of LLM request prompts. The regularization parameter λ is set to 0.001, and the exploration parameter ν in γ_t is set to 1. We also investigate the impact of delayed feedback, setting the maximum delay rounds (d_m) to 10 and 100in separate experiments. The reward obtained from selecting arm i at round t experiences a uniform random delay $d_{t,i}$ not exceeding d_m, meaning the reward is observed and used for neural network model training only after $d_{t,i}$ rounds. In each round's neural network model training, following the steps in Algorithm 3, the gradient descent step J is set to 10. Training is completed before proceeding to the next round.

For the real-world testbed experiments, we utilize the Alpaca dataset for LLM input requests [39], which contains 52 K examples generated in the style of self-instruct to serve as instruction-following demonstrations for fine-tuning language models. The LLM used for inference is Vicuna-7b-v1.5 [42], deployed on a server with NVIDIA GeForce RTX 4090 GPU simulating the edge. The processing time for each request on this hardware serves as the ground truth reward for the bandit framework. Similar to the simulated-reward experiments, $N = 20$ and $K = 4$, but each arm represents an LLM request, and its contextual vector is obtained by encoding the natural language of the request into a 768-dimensional embedding vector using BERT, so $d = 768$. To better fit real-world scenarios, we do not set explicit feedback delays. Instead, we test two groups of algorithms: synchronous training algorithms and asynchronous training algorithms. Synchronous training algorithms, like in the simulated-reward experiments, involve arm selection and a complete neural network model training process in each round, after which the next round begins. Asynchronous training, on the other hand, employs an additional single thread for asynchronous training, passively generating equivalent delayed feedback, of which the specific mechanism is detailed in Section III-D.

2) Baselines: We compare our proposed method against several representative baselines, which are grouped into two categories based on the training mechanism: synchronous and asynchronous training (i.e., with delayed feedback).

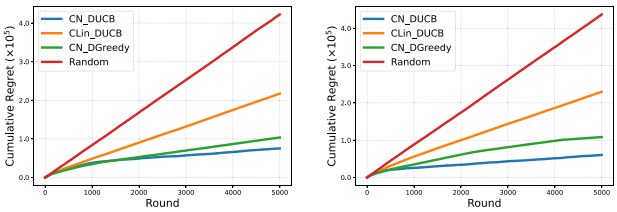
Synchronous Training Baselines: In the synchronous setting, where model training is completed within each round before the next begins, we consider the following algorithms:

- *CN_UCB:* A combinatorial neural Upper Confidence Bound (UCB) method that utilizes neural networks for scoring functions and assumes model parameters are updated every round [31].
- *CLin_UCB:* A combinatorial linear bandit algorithm based on a linear assumption for rewards [43].
- *CN_Greedy:* A neural-estimator-based ϵ-greedy method that selects arms based on the highest estimated rewards without an explicit exploration term.
- *Random:* A baseline that randomly selects requests for processing, serving as a lower bound for performance.

Asynchronous Training Algorithms: In the asynchronous setting, which involves delayed feedback, we evaluate our proposed method against the following baselines:

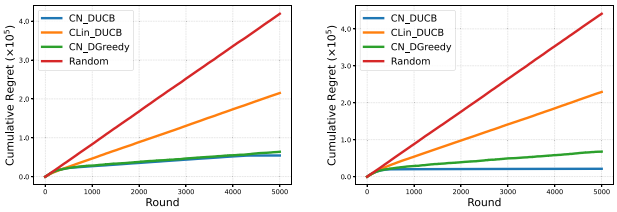
- *CN_DUCB (Ours):* Our proposed Combinatorial Neural Delayed-UCB method. It is designed to handle delayed feedback by adapting the neural UCB framework, making it suitable for real-world asynchronous training scenarios.
- *CLin_DUCB:* An adaptation of the CLin_UCB algorithm to handle delayed feedback, serving as a linear baseline in the asynchronous setting.
- *CN_DGreedy:* The adaptation of the CN_Greedy method for delayed feedback environments, which serves as a direct ϵ-greedy counterpart to our proposed algorithm.

3) Results: Simulated-Reward Experiments: The experimental results validate the superiority of our proposed CN_DUCB algorithm in handling non-linear rewards and delayed feedback. As illustrated in Figs. 4 and 5, CN_DUCB consistently achieves the lowest cumulative regret across both non-linear reward functions. This performance advantage is maintained under both short ($d_m = 10$) and long ($d_m = 100$) feedback delay conditions, demonstrating the algorithm's robustness. The significant regret incurred by CLin_DUCB highlights the inadequacy of linear models for these complex tasks. Moreover, CN_DUCB's consistent outperformance of CN_DGreedy underscores the effectiveness of its UCB-based exploration strategy over a simpler ϵ-greedy policy. Further analysis in Fig. 6(a) shows that neural network-based algorithms generally outperform the linear baseline in terms of average reward across various context dimensions d, confirming their



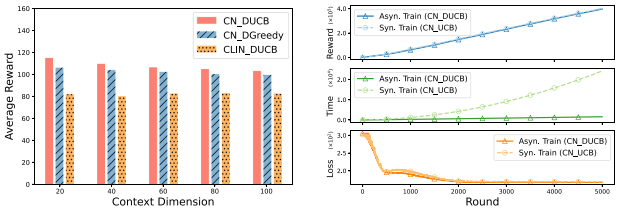
(a) $h_1(\boldsymbol{x}) = 10(\boldsymbol{x}^\top \boldsymbol{a})^2$ (b) $h_2(\boldsymbol{x}) = \boldsymbol{x}^\top A^\top A\boldsymbol{x}$

Fig. 4. Comparison of cumulative regret for different scheduling algorithms over 5000 rounds with synchronous training and a maximum feedback delay of 10 rounds ($d_m = 10$).



(a) $h_1(\boldsymbol{x}) = 10(\boldsymbol{x}^\top \boldsymbol{a})^2$ (b) $h_2(\boldsymbol{x}) = \boldsymbol{x}^\top A^\top A\boldsymbol{x}$

Fig. 5. Comparison of cumulative regret for different scheduling algorithms over 5000 rounds with synchronous training and an increased maximum feedback delay of 100 rounds ($d_m = 100$).

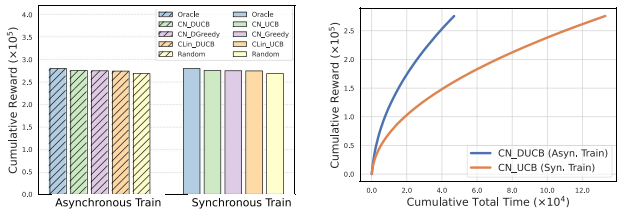


(a) Sync. train with $d_m = 10$ (b) No explicit d_m

Fig. 6. Performance comparison of scheduling algorithms under the non-linear reward function $h_1(x) = 10(x^\top a)^2$. Asynchronous training introduces implicit feedback delays.

scalability and effectiveness in high-dimensional spaces. The contribution of our asynchronous training design is demonstrated in Fig. 6(b), which directly compares the asynchronous training (CN_DUCB) with its synchronous version (CN_UCB). While CN_UCB obtains a marginally higher cumulative reward due to immediate information access, CN_DUCB operates with a dramatically lower cumulative time cost. This efficiency is achieved by decoupling arm selection from the blocking model training step. The convergence of the training loss for both methods confirms the stability and effectiveness of our asynchronous training mechanism. In summary, CN_DUCB presents a compelling performance trade-off that it achieves cumulative rewards comparable to the ideal synchronous case but at a fraction of the computational time, affirming its suitability for practical, time-sensitive applications.

Real-World Testbed Experiments: To validate our findings in a practical setting, we conduct experiments on a real-world testbed using the Alpaca dataset [39] and Vicuna-7B LLM [42]. The natural language prompts from the dataset are first transformed into contextual embedding vectors via a BERT model, which then served as the input for the online selection algorithms. The results, presented in Fig. 7, corroborate the conclusions drawn from the simulations and further underscore the practical



(a) Overall reward performance (b) Time-efficiency comparison

Fig. 7. Performance comparison on a real-world testbed using the Alpaca dataset, BERT encoder, and Vicuna LLM model.

benefits of our asynchronous approach. Fig. 7(a) displays the cumulative rewards for both asynchronous and synchronous algorithms. The results show that our CN_DUCB achieves a cumulative reward nearly identical to the synchronous CN_UCB and approaches the performance of the Oracle upper bound. This finding demonstrates that our asynchronous method can operate with virtually no loss in reward performance in a realistic environment with inherent system delays. Both neural network approaches substantially outperform their linear and ϵ-greedy counterparts, confirming their effectiveness on real-world contextual data. The advantage in efficiency is highlighted in Fig. 7(b), which plots cumulative reward against total wall-clock time. CN_DUCB demonstrates a significantly higher reward-to-time efficiency, achieving the same reward level as CN_UCB in a fraction of the time. This improvement stems from eliminating the model training latency that acts as a major bottleneck in the synchronous system. The results conclusively show that our asynchronous framework is not only effective in terms of decision-making quality but is also a far more efficient and practical solution for real-world LLM inference scheduling.

C. Performance of Edge-Cloud Scheduling

This section details the performance evaluation of our OCE-CRS framework against several baselines. We conduct extensive experiments using a custom-built simulator that models a realistic cloud-edge environment to assess throughput, JCT, and queueing delay under various system loads. The results demonstrate that our learning-based approach, particularly OCE-CRS with CN_DUCB, significantly outperforms traditional and heuristic-based scheduling policies.

1) Experimental Setup: Our evaluation is performed in a simulated cloud-edge system configured to reflect real-world characteristics. For all learning-based schedulers, the key hyperparameters were kept consistent with the experiments in Section VI-B: the regularization parameter λ was set to 0.001, and the exploration parameter ν in the UCB calculation was set to 1.

Hardware Environment: The edge server is modeled after an NVIDIA GeForce RTX 4090 GPU, capable of processing one batch of requests at a time with a batch capacity of $K = 4$ requests. The cloud server is configured with higher computational power, reflected by a processing time multiplier of 0.7 relative to the edge, and can handle $C_{cloud} = 8$ concurrent individual requests. A network round-trip time (RTT) of 50ms is added for all requests offloaded to the cloud. In essence, the edge server utilizes a dynamic batching model to process requests collectively, while the cloud server processes them as concurrent, individual tasks without batching.

TABLE II
OVERALL PERFORMANCE COMPARISON OF OCE-CRS AND BASELINE SCHEDULERS UNDER A STATIC BATCHING MODEL

Scheduling Plan	Total TP (rps)		Edge TP (rps)		Avg. JCT (s)		Avg. QD (s)	
	Qwen	Vicuna	Qwen	Vicuna	Qwen	Vicuna	Qwen	Vicuna
Random_Offload	3.6165	2.7993	0.5163	0.2705	727.30	1101.57	723.98	1097.31
Edge_Cloud_FCFS	3.6011	2.7647	0.5262	0.2826	942.10	1559.03	938.77	1554.71
Least_Connections	3.6008	2.7653	0.5251	0.2827	935.23	1558.79	931.90	1554.47
Queue_Length_Based	3.6014	2.7653	0.5290	0.2893	937.80	1561.78	934.48	1557.46
Round_Robin_Offload	3.6050	2.7656	0.5268	0.2942	933.24	1564.57	929.92	1560.26
Historical_Least_Response_Time	3.7163	2.8378	0.4069	0.3004	132.82	140.52	129.74	136.31
OCE-CRS (with Clin_DUCB)	3.9280	3.0184	0.5430	0.2761	77.43	52.90	74.38	48.94
OCE-CRS (with CN_DGreedy)	4.3609	3.8951	0.5151	0.3444	53.24	57.36	50.49	54.30
OCE-CRS (with CN_DUCB)	4.3588	3.8956	0.5315	0.3501	43.87	45.91	41.12	42.84

Workload and Dataset: LLM requests are drawn from the Alpaca dataset [39], with prompts processed by Qwen2.5-3B-Instruct [44] and Vicuna-7b-v1.5 [42] models. The ground truth processing time for each request is based on real-world inference measurements. Request arrivals are simulated following a Poisson process with a configurable rate λ, where $\lambda = 5$ by default.

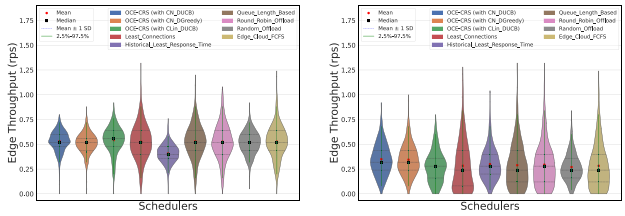
Embeddings: Prompts are encoded into 768-dimensional feature vectors using a pre-trained BERT-base model, which serves as the context for our learning-based schedulers.

Scheduling and Queueing Logic: Upon arrival, requests enter a central waiting queue managed by the active scheduler. For heuristic baselines, this is a standard FCFS queue. For our learning-based OCE-CRS framework, requests are organized in a priority queue based on their algorithm-specific scores (e.g., predicted processing time or UCB score). The scheduling process is triggered by new arrivals or task completions. The core OCE-CRS logic first selects a candidate pool of high-priority requests; from this pool, the top-K scoring requests are assigned to the edge for batched execution, while the remaining candidates in the pool are offloaded to the cloud for individual, concurrent processing, subject to slot availability.

2) Baselines: We compare the performance of our proposed OCE-CRS framework (using CN_DUCB, CN-DGreedy, and CLin-DUCB) against a suite of representative schedulers:

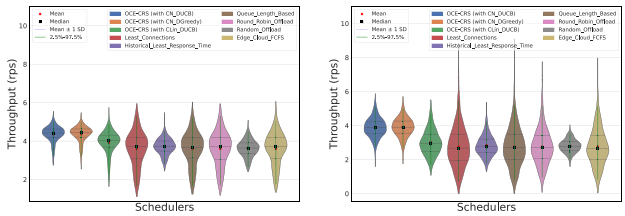
- *Edge-Cloud FCFS:* A basic First-Come-First-Served policy where requests are sent to the edge; if the edge is full, they are offloaded to the cloud.
- *Random Offload:* Randomly assigns incoming requests to either the edge or the cloud.
- *Round Robin Offload:* Alternates assignments between the edge and the cloud in a round-robin fashion [45].
- *Least Connections:* Assigns requests to the node (edge or cloud) with the fewest active jobs [46].
- *Historical Least Response Time:* Directs requests to the node predicted to offer the lowest Job Completion Time, based on the historical average processing times of each node and their current estimated queueing delay [47].
- *Queue Length Based:* Offloads requests to the node with the shorter queue, where queue length is measured by the number of busy processing slots [48].

3) Results and Analysis: Our extensive evaluations demonstrate that the proposed OCE-CRS framework, particularly when integrated with the CN_DUCB algorithm, achieves significant performance gains across all key metrics compared to traditional and heuristic-based baselines. The results are consistent across both Qwen and Vicuna language models, underscoring the robustness and generalizability of our approach.



(a) Qwen model (b) Vicuna model

Fig. 8. Distribution of instantaneous edge throughput for different scheduling policies under a static batching model.

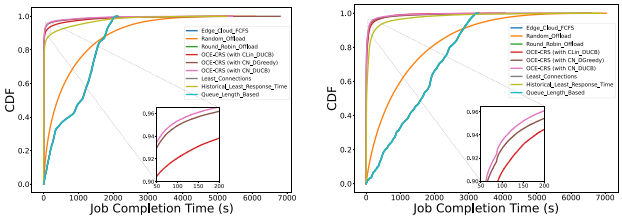


(a) Qwen model (b) Vicuna model

Fig. 9. Distribution of instantaneous total system throughput (edge and cloud combined) for different scheduling policies under a static batching model.

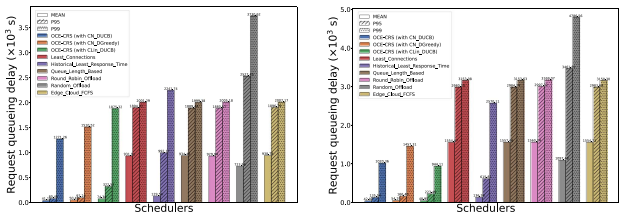
Overall Performance: A comprehensive summary of the results is presented in Table II. OCE-CRS (with CN_DUCB) consistently emerges as the top-performing scheduler, delivering the highest total and edge throughput, while simultaneously achieving the lowest average job completion time (JCT) and queueing delay (QD). For instance, with the Vicuna model, it reduces the average JCT by 67% compared to the best-performing baseline, Historical_Least_Response_Time, and by over 97% compared to Edge_Cloud_FCFS. This highlights the impact of our learning-based scheduling policy on system efficiency. Among our proposed methods, the UCB-based exploration of CN_DUCB proves superior to the ϵ-greedy policy of CN_DGreedy and the linear assumption of CLin_DUCB.

Throughput Analysis: Figs. 8 and 9 illustrate the distribution of instantaneous edge and total system throughput, respectively.



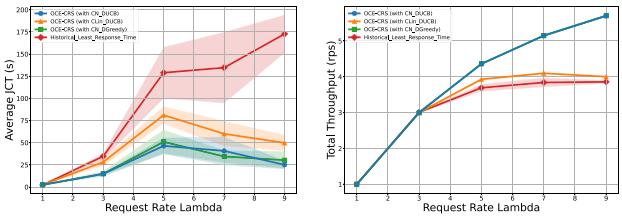
(a) Qwen model (b) Vicuna model

Fig. 10. CDF of JCT for scheduling algorithms under static batching.



(a) Qwen model (b) Vicuna model

Fig. 11. Request queueing delay for different schedulers under a static batching model, measured in thousands of seconds ($\times 10^3$ s). The bars show the mean, 95th percentile (P95), and 99th percentile (P99) delays.



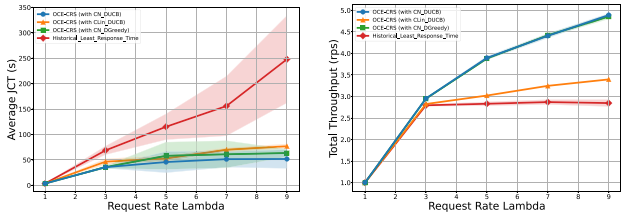
(a) JCT (b) Total throughput

Fig. 12. System performance under varying request arrival rates (λ) for the Qwen model.

The violin plots show that OCE-CRS schedulers, especially the CN_DUCB variant, not only achieve a higher mean throughput but also exhibit a more favorable distribution, with a greater density of samples concentrated at higher throughput rates. This indicates a more consistent and stable system performance, effectively utilizing both edge and cloud resources to maximize request processing capacity.

Latency Reduction: The Cumulative Distribution Function (CDF) of JCT in Fig. 10 shows that OCE-CRS methods complete a significantly larger fraction of jobs in a shorter amount of time. The inset plot reveals that over 94% of jobs finish within 100 seconds under OCE-CRS (with CN_DUCB), a mark that baselines like Least_Connections fail to reach for even 90% of jobs. Furthermore, Fig. 11 provides a detailed breakdown of queueing delays, where the high delay values for baseline schedulers are an intentional result of the high-concurrency simulation designed to stress-test the system. OCE-CRS drastically reduces not only the mean delay but also the crucial 95th (P95) and 99th (P99) percentile tail latencies. This reduction in tail latency is critical for ensuring a responsive and high-quality user experience, preventing requests from experiencing excessively long waits.

Scalability and Robustness: To evaluate robustness under dynamic conditions, we vary the request arrival rate, λ. As shown in Figs. 12 and 13, our OCE-CRS (with CN_DUCB)



(a) JCT (b) Total throughput

Fig. 13. System performance under varying request arrival rates (λ) for the Vicuna model.

demonstrates superior scalability. As system load increases, its average JCT rises much more slowly compared to all other methods. Concurrently, it sustains a higher total throughput, indicating its ability to handle heavier traffic without saturation. In contrast, the performance of heuristic baselines like Historical_Least_Response_Time degrades rapidly under high load, highlighting the adaptability and forward-looking nature of our learning-based scheduler.

Performance Analysis: OCE-CRS outperforms baselines by mitigating HOL blocking and allocating resources in a content-aware manner. FCFS is vulnerable to long jobs stalling the queue, which inflates latency and suppresses throughput. OCE-CRS predicts runtimes online and applies an SJF-style rule, sending short jobs to the edge and offloading computationally intensive ones to the cloud. This policy reduces average JCT and queueing delay (Figs. 10 and 11) and enhances edge throughput (Fig. 8). The combination of proactive HOL avoidance and intelligent offloading alleviates the main bottleneck and explains the consistent gains over the baselines.

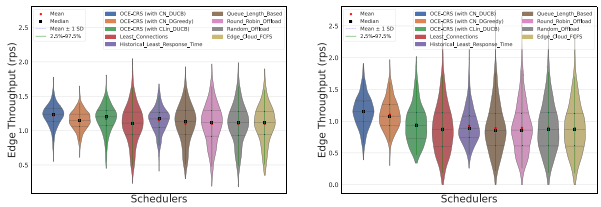
D. Performance of Continuous Batching

To rigorously assess our proposed scheduling algorithms, we evaluate them within a simulated environment that functionally mirrors the high-throughput capabilities of modern inference systems. Advanced serving engines, such as Orca [17] and vLLM [18], employ sophisticated techniques like continuous batching to maximize hardware utilization. Faithfully simulating these systems at the iteration level is exceedingly complex. Therefore, our evaluation adopts a high-level functional abstraction of this paradigm. Instead of modeling the micro-level dynamics, we simulate the macro-level outcome: a system capable of processing multiple requests concurrently. This approach allows us to isolate and test the core logic of the scheduling policies themselves, providing a clear and direct measure of their effectiveness in a high-throughput context. The results validate that our proposed framework excels in this abstracted environment, indicating its strong potential for practical application in real-world, high-performance systems.

1) Batching Paradigms and Our Simulation Approach: The choice of a batching strategy greatly influences serving performance. A traditional approach is static batching, where a fixed group of requests are processed together as a single unit. This can cause delays, as short jobs are forced to wait for longer ones within the same batch. More advanced systems utilize continuous batching, an iteration-level technique that allows requests to dynamically join or leave the batch, thereby maximizing hardware utilization. Our simulation evaluates scheduling policies using a functional abstraction of this advanced model.

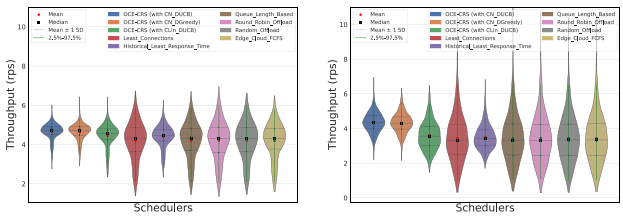
TABLE III
OVERALL PERFORMANCE COMPARISON OF OCE-CRS AND BASELINE SCHEDULERS IN A CONTINUOUS BATCHING ENVIRONMENT

Scheduling Plan	Total TP (rps)		Edge TP (rps)		Avg. JCT (s)		Avg. QD (s)	
	Qwen	Vicuna	Qwen	Vicuna	Qwen	Vicuna	Qwen	Vicuna
Random_Offload	4.1975	3.4334	1.0968	0.8955	574.11	1145.82	571.26	1142.33
Edge_Cloud_FCFS	4.1945	3.4327	1.0990	0.8880	573.01	1141.35	570.15	1137.87
Least_Connections	4.1972	3.4328	1.0977	0.8873	568.07	1148.84	565.21	1145.36
Queue_Length_Based	4.1965	3.4334	1.1016	0.8906	558.45	1148.34	555.60	1144.85
Round_Robin_Offload	4.1969	3.4324	1.0987	0.8945	564.43	1139.58	561.57	1136.10
Historical_Least_Response_Time	4.4038	3.5032	1.1525	0.9260	31.14	67.92	28.42	64.51
OCE-CRS (with Clin_DUCB)	4.4505	3.6102	1.1719	0.9528	91.48	57.62	88.79	54.31
OCE-CRS (with CN_DGreedy)	4.6365	4.3215	1.1480	1.1019	73.09	96.22	70.51	93.45
OCE-CRS (with CN_DUCB)	4.6546	4.3592	1.2203	1.1522	44.94	37.94	42.36	35.20



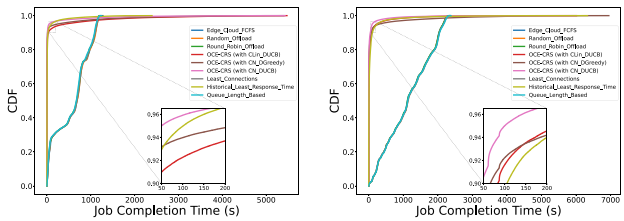
(a) Qwen model (b) Vicuna model

Fig. 14. Distribution of instantaneous edge throughput for different scheduling policies within a simulated continuous batching environment.



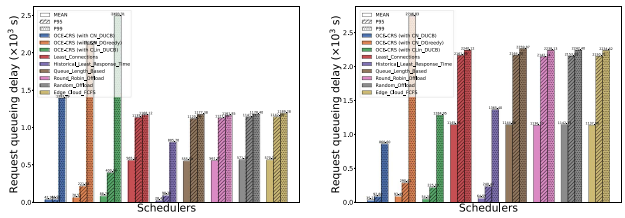
(a) Qwen model (b) Vicuna model

Fig. 15. Distribution of instantaneous total system throughput for different schedulers in a simulated continuous batching environment.



(a) Qwen model (b) Vicuna model

Fig. 16. CDF of JCT for scheduling algorithms under continuous batching.



(a) Qwen model (b) Vicuna model

Fig. 17. Request queueing delay for different schedulers in a simulated continuous batching environment, measured in thousands of seconds ($\times 10^3$ s). The bars show the mean, 95th (P95), and 99th percentile (P99) delays.

We simulate the server with K parallel processing slots, representing the result of a continuous batching engine with an effective concurrency of K. This abstracted model is valuable because it allows us to rigorously test the core scheduling logic, deciding which requests to prioritize, without the complexity of a low-level implementation. The success of a scheduler in this model is therefore a strong indicator of its effectiveness in a real-world system.

2) Results and Analysis: The aggregated results in Table III show that OCE-CRS (with CN_DUCB) remains the dominant scheduling strategy. It achieves the highest throughput in Figs. 14 and 15 while simultaneously recording the lowest average JCT and average queueing delay in Figs. 16 and 17. When compared with the static batching results, it is evident that the continuous batching model improves performance for nearly all schedulers. However, our learning-based framework, especially the CN_DUCB variant, most effectively exploits this increased system potential, maintaining and even widening its performance gap over the heuristic baselines.

However, Fig. 17 reveals that despite their strong average performance, OCE-CRS schedulers suffer from elevated P99 queueing delays due to persistent de-prioritization of a small number of requests with poor predicted outcomes, which may be repeatedly bypassed by newer arrivals. This highlights a trade-off between optimizing overall efficiency and ensuring worst-case fairness. Fortunately, this issue can be readily mitigated in practice through incorporating a simple aging-based priority adjustment, where the priority of delayed requests gradually increases over time. This approach ensures fairness and prevents long delays for any individual request, without compromising the overall throughput, making it a practical and effective solution.

3) Adaptability: The results from the continuous batching evaluation validate the efficacy and adaptability of the OCE-CRS framework. Even in a more efficient and flexible system environment, the intelligent learning-based decision-making of

OCE-CRS provides a distinct and significant advantage over traditional scheduling policies, solidifying its position as a superior solution for complex edge-cloud scheduling tasks.

VII. Conclusion

In this paper, we address the critical challenge of efficiently scheduling LLM requests in cloud-edge collaborative systems. We introduce the OCE-CRS framework, which enables an SJF-inspired policy that prioritizes shorter requests for the edge, thereby simultaneously optimizing resource utilization to enhance system throughput and mitigating HOL blocking to improve responsiveness. Its asynchronous training strategy prevents time-consuming model updates from stalling scheduling decisions by decoupling the two processes and effectively managing the delayed application of feedback. The robustness of the CN_DUCB algorithm is supported by a theoretical analysis demonstrating a sublinear regret bound. Finally, extensive experiments validate the effectiveness of our method, highlighting that its superiority becomes most pronounced under moderate to high system loads. For instance, in a static batching simulation where high request rates caused traditional methods' latency to degrade rapidly, our framework boosts total system throughput by over 21% while simultaneously reducing the average JCT by up to 95% compared to a standard FCFS policy. This significant latency improvement is achieved while also maintaining higher total system throughput. Furthermore, this adaptability under load is consistently demonstrated in a more advanced continuous batching environment, confirming the framework's practical value for modern, high-throughput LLM serving systems.

While our framework demonstrates strong performance for text-based LLM requests, which represent a majority of current workloads, we acknowledge that modern LLMs are increasingly multimodal, processing combined text-and-image inputs. We believe our framework is extensible to this important domain. The core "encoder-predictor" architecture can be adapted by replacing the current text-only encoder (BERT) with a multimodal encoder that fuses features from both visual and textual data into a single embedding vector. This vector can then be fed into the same neural network predictor without altering the fundamental bandit algorithm, UCB logic, or the asynchronous training mechanism. However, predicting processing times for multimodal inputs presents new challenges, as the model must learn more complex relationships between input characteristics (e.g., image resolution, content complexity) and execution time. Extending and evaluating our online learning framework for the efficient scheduling of multimodal LLM requests is a promising and significant direction for future research.

References

[1] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proc. IEEE*, vol. 107, no. 8, pp. 1655–1674, Aug. 2019.

[2] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surv. Tut.*, vol. 23, no. 4, pp. 2131–2165, Fourth Quarter 2021.

[3] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter 2017.

[4] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1584–1607, Aug. 2019.

[5] C. Kai, H. Zhou, Y. Yi, and W. Huang, "Collaborative cloud-edge-end task offloading in mobile-edge computing networks with limited communication capability," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 2, pp. 624–634, Jun. 2021.

[6] A. Naouri, H. Wu, N. A. Nouri, S. Dhelim, and H. Ning, "A novel framework for mobile-edge computing by optimizing task offloading," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 13065–13076, Aug. 2021.

[7] W. Fan et al., "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 238–256, Jan. 2024.

[8] Z. Yao, Z. Tang, J. Lou, P. Shen, and W. Jia, "VELO: A vector database-assisted cloud-edge collaborative LLM QoS optimization framework," 2024, *arXiv:2406.13399*.

[9] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *Proc. 14th USENIX Symp. Netw. Syst. Des. Implementation*, 2017, pp. 613–627.

[10] A. Gujarati et al., "Serving {DNNs} like clockwork: Performance predictability from the bottom up," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation*, 2020, pp. 443–462.

[11] Y. Laili, F. Guo, L. Ren, X. Li, Y. Li, and L. Zhang, "Parallel scheduling of large-scale tasks for industrial cloud–edge collaboration," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3231–3242, Feb. 2023.

[12] L. Qin, S. Chen, and X. Zhu, "Contextual combinatorial bandit and its application on diversified online recommendation," in *Proc. SIAM Int. Conf. Data Mining*, SIAM, 2014, pp. 461–469.

[13] S. Li, B. Wang, S. Zhang, and W. Chen, "Contextual combinatorial cascading bandits," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2016, pp. 1245–1253.

[14] T. Wolf and M. A. Franklin, "Locality-aware predictive scheduling of network processors," in *IEEE Int. Symp. Perform. Anal. Syst. Softw.*, Citeseer, 2001, pp. 152–159.

[15] W. Cui et al., "Enable simultaneous DNN services based on deterministic operator overlap and precise latency prediction," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021, pp. 1–15.

[16] Y. Li, J. Guo, Y. Li, T. Wang, and W. Jia, "Adversarial bandits with multi-user delayed feedback: Theory and application," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9383–9397, Oct. 2024.

[17] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proc. 16th USENIX Symp. Operating Syst. Des. Implementation*, 2022, pp. 521–538.

[18] W. Kwon et al., "Efficient memory management for large language model serving with pagedattention," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 611–626.

[19] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, "Fast distributed inference serving for large language models," 2023, *arXiv:2305.05920*.

[20] H. Qiu et al., "Efficient interactive llm serving with proxy model-based sequence length prediction," 2024, *arXiv:2404.08509*.

[21] J. Cai, W. Liu, Z. Huang, and F. R. Yu, "Task decomposition and hierarchical scheduling for collaborative cloud-edge-end computing," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 4368–4382, Nov./Dec. 2024.

[22] J. Liu et al., "Edge-cloud collaborative computing on distributed intelligence and model optimization: A survey," 2025, *arXiv:2505.01821*.

[23] G. Nieto, I. de la Iglesia, U. Lopez-Novoa, and C. Perfecto, "Deep reinforcement learning techniques for dynamic task offloading in the 5G edge-cloud continuum," *J. Cloud Comput.*, vol. 13, no. 1, 2024, Art. no. 94.

[24] J. Wang et al., "Toward scalable generative AI via mixture of experts in mobile edge networks," 2024, *arXiv:2402.06942*.

[25] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large language models (LLMs) inference offloading and resource allocation in cloud-edge computing: An active inference approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11253–11264, Dec. 2024.

[26] J. Mai, K. Cao, and T. Wei, "Personalized federated learning with state-adaptive IoT device scheduling in mobile edge computing," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, to be published, doi: 10.1109/TCAD.2025.3560221.

[27] W. Chen, W. Hu, F. Li, J. Li, Y. Liu, and P. Lu, "Combinatorial multi-armed bandit with general reward functions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1659–1667.

[28] L. Chen and J. Xu, "Task replication for vehicular cloud: Contextual combinatorial bandit with delayed feedback," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 748–756.

[29] C.-S. Yang, R. Pedarsani, and A. S. Avestimehr, "Edge computing in the dark: Leveraging contextual-combinatorial bandit and coded computing," *IEEE/ACM Trans. Netw.*, vol. 29, no. 3, pp. 1022–1031, Jun. 2021.

[30] D. Zhou, L. Li, and Q. Gu, "Neural contextual bandits with UCB-based exploration," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 11492–11 502.

[31] T. Hwang, K. Chai, and M.-H. Oh, "Combinatorial neural bandits," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 14203–14236.

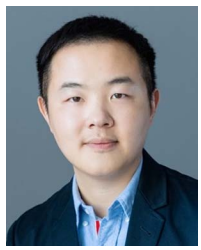
[32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
[33] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 8580–8589.
[34] Y. Cao and Q. Gu, "Generalization bounds of stochastic gradient descent for wide and deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019.
[35] Z. Allen-Zhu, Y. Li, and Z. Song, "A convergence theory for deep learning via over-parameterization," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 242–252.
[36] L. Yang, "Contextual bandits in imperfect environments: Analysis and applications," Dissertations & Theses, Univ. California, Riverside, 2021.
[37] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, pp. 2312–2320.
[38] B. Lu, J. Yang, J. Xu, and S. Ren, "Improving QoE of deep neural network inference on edge devices: A bandit approach," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 21409–21420, Nov. 2022.
[39] R. Taori et al., "Stanford alpaca: An instruction-following LLaMA model," *GitHub Repository*, 2023. [Online]. Available: https://github.com/tatsu-lab/stanford_alpaca
[40] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," 2024, *arXiv:2404.14219*.
[41] A. Q. Jiang et al., "Mistral 7B," 2023, *arXiv:2310.06825*.
[42] L. Zheng et al., "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 46595–46623.
[43] Z. Wen, B. Kveton, and A. Ashkan, "Efficient learning in large-scale combinatorial semi-bandits," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2015, pp. 1113–1122.
[44] Q. Team, "Qwen2 technical report," 2024, *arXiv:2412.15115*.
[45] G. Miao, J. Zander, K. W. Sung, and S. B. Slimane, *Fundamentals of Mobile Data Networks*. Cambridge, U.K.: Cambridge Univ. Press, 2016.
[46] T. W. Harjanti, H. Setiyani, and J. Trianto, "Load balancing analysis using round-robin and least-connection algorithms for server service response time," *Appl. Technol. Comput. Sci. J.*, vol. 5, no. 2, pp. 119–128, 2022.
[47] A. Silberschatz, J. L. Peterson, and P. B. Galvin, *Operating System Concepts*. Reading, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1991.
[48] E. J. Ghomi, A. M. Rahmani, and N. N. Qader, "Load-balancing algorithms in cloud computing: A survey," *J. Netw. Comput. Appl.*, vol. 88, pp. 50–71, 2017.

Yandi Li received the BE degree from the School of Optoelectronic Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2013. He is currently working toward the PhD degree with the Guangdong Key Lab of AI and Multi-Modal Data Processing, Department of Computer Science, Beijing Normal-Hong Kong Baptist University, Zhuhai, China. He is supervised by Dr. Jianxiong Guo, and his research interests include social networks, online algorithms, edge computing, and machine learning.

Jianxiong Guo (Member, IEEE) received the BE degree from the School of Chemistry and Chemical Engineering, South China University of Technology, Guangzhou, China, in 2015, and the PhD degree from the Department of Computer Science, University of Texas at Dallas, Richardson, TX, USA, in 2021. He is currently an associate professor with the Advanced Institute of Natural Sciences, Beijing Normal University, Zhuhai, China. He is a member of ACM/CCF. He has published more than 100 peer-reviewed papers and has been a reviewer for many famous international journals/conferences. His research interests include social networks, wireless sensor networks, combinatorial optimization, and machine learning.

Zhiqing Tang (Member, IEEE) received the BS degree from the School of Communication and Information Engineering, University of Electronic Science and Technology of China, China, in 2015 and the PhD degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in 2022. He is currently an assistant professor with the Advanced Institute of Natural Sciences, Beijing Normal University, China. His current research interests include edge computing, resource scheduling, and reinforcement learning.

Xingjian Ding received the BE degree in electronic information engineering from Sichuan University, Chengdu, China, in 2012, the MS degree in software engineering from Beijing Forestry University, Beijing, China, in 2017, and the PhD degree from the School of Information, Renmin University of China, Beijing, in 2021. He is currently an assistant professor with the School of Software Engineering, Beijing University of Technology, Beijing. His research interests include wireless rechargeable sensor networks, approximation algorithms design and analysis, and blockchain.



Juncheng Wang (Member, IEEE) received the BEng degree in electrical engineering from Shanghai Jiao Tong University, Shanghai, China, in 2014, the MSc degree in electrical and computer engineering from the University of Alberta, Edmonton, AB, Canada, in 2017, and the PhD degree in electrical and computer engineering from the University of Toronto, Toronto, ON, Canada, in 2023. He is currently an assistant professor with the Department of Computer Science, Hong Kong Baptist University, Hong Kong, China. His research interests include network artificial intelligence, distributed machine learning, and online optimization.

Tian Wang (Senior Member, IEEE) received the BSc and MSc degrees in computer science from Central South University, in 2004 and 2007, respectively, and the PhD degree in computer science from the City University of Hong Kong, in 2011. Currently, he is a professor with the Institute of Artificial Intelligence and Future Networks, Beijing Normal University. His research interests include Internet of Things, edge computing, and mobile computing. He has 30 patents and has published more than 200 papers in high-level journals and conferences. He was a co-recipient of the Best Paper Runner-up Award of IEEE/ACM IWQoS 2024. He has more than 16000 citations, according to Google Scholar. His H-index is 74.

Weijia Jia (Fellow, IEEE) received the BSc and MSc degrees from Center South University, China, in 1982 and 1984, respectively and the master of applied Sci. and PhD degrees from the Polytechnic Faculty of Mons, Belgium, in 1992 and 1993, respectively, all in computer science. He is currently a chair professor, director with the BNU-UIC Institute of Artificial Intelligence and Future Networks, Beijing Normal University (Zhuhai) and VP for Research of BNU-HKBU United International College (UIC) and has been the Zhiyuan chair professor with Shanghai Jiao Tong University, China. He was the chair professor and the deputy director with the State Kay Laboratory of Internet of Things for Smart City, University of Macau. From 93-95, he joined German National Research Center for Information Science (GMD) in Bonn (St. Augustine) as a research fellow. From 95-13, he worked with the City University of Hong Kong as a professor. His contributions have been recognized as optimal network routing and deployment; anycast and QoS routing, sensors networking, AI (knowledge relation extractions; NLP, etc.), and edge computing. He has more than 600 publications in the prestige international journals/conferences and research books and book chapters. He has received the best product awards from the International Science & Tech. Expo (Shenzhen) in 20112012 and the 1st Prize of Scientific Research Awards from the Ministry of Education of China in 2017 (list 2). He has served as area editor for various prestige international journals, chair, PC member, and keynote speaker for many top international conferences. He is the distinguished member of CCF.